

SNAAR TekMess

Arquitectura y Patrones de Diseño

Proyecto: Sistema de Autenticación, Autorización y Reporte (SNAAR) de TekMess

Versión: 1.0

Fecha: 15 de junio de 2026

Asignatura: Análisis y Diseño de Software

Tabla de Contenidos

- [1. Introducción](#)
- [2. Arquitectura de Tres Capas](#)
 - 2.1 Definición
 - 2.2 Estructura de Carpetas
 - 2.3 Justificación
- [3. Patrones de Diseño Implementados](#)
 - 3.1 Patrón Command
 - 3.2 Patrón Observer
 - 3.3 Patrón DAO
 - 3.4 Patrón Bridge
- [4. Patrones de Validación y Seguridad](#)
- [5. Generación de Credenciales](#)
- [6. Reportes \(RF-03V2\)](#)
- [7. Gestión de Sesiones y Autenticación](#)
- [8. Ventajas de la Arquitectura](#)
- [9. Decisiones Tecnológicas](#)
- [10. Riesgos y Mitigaciones](#)
- [11. Conclusión](#)
- [12. Referencias](#)

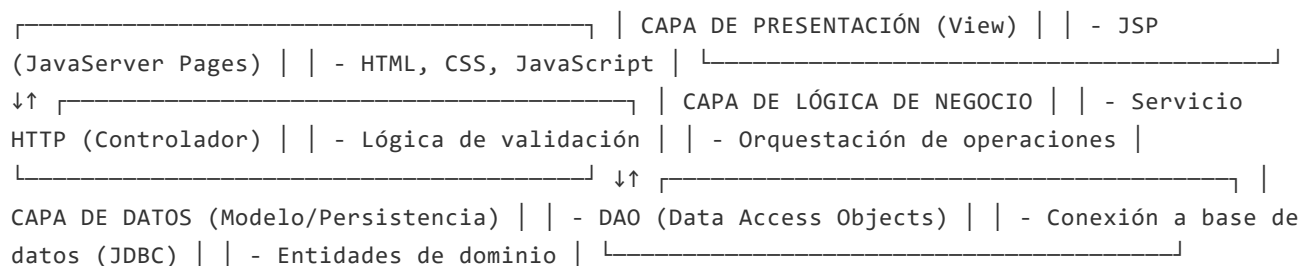
1. Introducción

El sistema SNAAR implementa una arquitectura de **tres capas (3-Tier Architecture)** combinada con patrones de diseño específicos para garantizar mantenibilidad, escalabilidad y flexibilidad. Este documento justifica cada decisión arquitectónica y de diseño tomada.

2. Arquitectura de Tres Capas (3-Tier Architecture)

2.1 Definición

La arquitectura se organiza en tres capas independientes:



2.2 Estructura de Carpetas

```
src/main/java/com/tekmess/snaar/ ├── controlador/ | ├── http/ ◀ Servlets/Controladores HTTP
| | ├── AuthController.java | | ├── EmpleadoController.java | | └── ... | └── servicio/ ◀
Servicios de negocio | ├── AuthService.java | ├── EmpleadoServicio.java | └──
ReporteServicio.java ├── modelo/ | ├── dao/ ◀ Data Access Objects | | ├── IEmpleadoDAO.java
| | ├── EmpleadoDAO.java | | └── ... | └── entidad/ ◀ Entidades de dominio | ├──
Empleado.java | ├── Usuario.java | └── ... ├── patron/ | ├── command/ ◀ Patrón Command | |
| ├── IComando.java | | ├── CrearEmpleadoComando.java | | └── ... | └── observer/ ◀ Patrón
Observer | ├── IObservador.java | ├── ObservadorAuditoria.java | └── ... └── util/ ├──
CifradorContrasena.java ├── ConexionBD.java └── ValidadorDatos.java
```

2.3 Justificación de la Arquitectura de Tres Capas

Ventajas:

1. **Separación de responsabilidades:** Cada capa tiene una responsabilidad única y bien definida.
 - La capa de presentación **NO** accede directamente a la base de datos.
 - La capa de lógica de negocio **NO** conoce detalles de la presentación.
2. **Mantenibilidad:** Un cambio en la base de datos no afecta la interfaz de usuario.
 - Si se cambia de SQL a NoSQL, solo cambia la capa de datos.
 - Si se rediseña la UI, solo cambia la capa de presentación.
3. **Testabilidad:** Cada capa puede testarse independientemente.

- Se pueden crear mocks de la capa de datos para testear servicios.
- Se pueden testear servicios sin una base de datos real.

4. **Escalabilidad:** Fácil agregar nuevas funcionalidades sin afectar capas existentes.

- Nuevos controladores se añaden sin modificar DAOs.
- Nuevos servicios se crean sin tocar vistas.

5. **Reutilización:** La capa de servicios puede ser usada por múltiples controladores.

- `EmpleadoServicio` puede ser usado por controladores HTTP, tareas programadas y APIs internas.

Desventajas y Mitigación:

Desventaja	Mitigación
Más capas = más archivos	Uso claro de interfaces (I-prefixed) y convenciones de nombrado
Overhead de comunicación entre capas	En sistemas pequeños, el impacto es negligible
Curva de aprendizaje	Documentación clara y patrones consistentes

3. Patrones de Diseño Implementados

3.1 Patrón Command

Ubicación en el código:

```
patron/command/ ├── IComando.java ← Interfaz base ├── CrearEmpleadoComando.java ← Comando  
concreto ├── EditarEmpleadoComando.java ├── EliminarEmpleadoComando.java ├──  
InvocadorOperaciones.java ← Invocador └── ResultadoComando.java ← Resultado
```

Definición:

El patrón Command encapsula una solicitud como un objeto, permitiendo parametrizar clientes con diferentes solicitudes, encolar solicitudes y registrar el historial de operaciones.

Implementación en SNAAR:

```
// Interfaz Command
public interface IComando {
    ResultadoComando ejecutar();
}

// Comando concreto: Crear empleado
public class CrearEmpleadoComando implements IComando {
    private Empleado empleado;
    private EmpleadoServicio servicio;

    @Override
    public ResultadoComando ejecutar() {
        // Lógica de creación encapsulada
        return servicio.crearEmpleado(empleado);
    }
}

// Invocador: Gestiona comandos
public class InvocadorOperaciones {
    private Stack<IComando> historial = new Stack<>();

    public void ejecutar(IComando comando) {
        ResultadoComando resultado = comando.ejecutar();
        if (resultado.esExitoso()) {
            historial.push(comando); // Registra en historial
        }
    }
}
```

Justificación:

1. **Encapsulación de operaciones:** Cada operación (crear, editar, eliminar) es independiente.
2. **Historial de operaciones:** Facilita la auditoría y el deshacer (`deshacerUltimaOperacion`).
3. **Extensibilidad:** Agregar nuevas operaciones solo requiere implementar `IComando` .
4. **Testing:** Cada comando puede testearse aisladamente.

Casos de uso:

- RF-01.01: Crear empleado → `CrearEmpleadoComando`
- RF-01.02: Editar empleado → `EditarEmpleadoComando`
- RF-01.03: Eliminar empleado → `EliminarEmpleadoComando`
- Deshacer última operación → `InvocadorOperaciones.deshacer()`

3.2 Patrón Observer

Ubicación en el código:

```
patron/observer/ ├── ISujeto.java ← Sujeto observable ├── IObservador.java ← Observador ├──
SujetoGestionEmpleado.java ← Sujeto concreto ├── ObservadorAuditoria.java ← Observador
concreto └── ObservadorNotificacion.java
```

Definición:

El patrón Observer define una relación uno-a-muchos entre objetos, de modo que cuando un objeto cambia de estado, todos sus dependientes son notificados automáticamente.

Implementación en SNAAR:

```
// Interfaz Sujeto
public interface ISujeto {
    void agregarObservador(IObservador obs);
    void eliminarObservador(IObservador obs);
    void notificarObservadores(EventoSistema evento);
}

// Sujeto concreto
public class SujetoGestionEmpleado implements ISujeto {
    private List<IObservador> observadores = new ArrayList<>();

    @Override
    public void notificarObservadores(EventoSistema evento) {
        for (IObservador obs : observadores) {
            obs.actualizar(evento); // Notifica a todos
        }
    }
}

// Observador concreto: Auditoría
public class ObservadorAuditoria implements IObservador {
    @Override
    public void actualizar(EventoSistema evento) {
        registrarAuditoria(evento.getTipo(), evento.getDetalles());
    }
}
```

Justificación:

1. **Desacoplamiento:** La lógica de auditoría no está mezclada con creación de empleados.
2. **Extensibilidad:** Agregar nuevos observadores (SMS, email) sin modificar el sujeto.

3. **Múltiples reacciones:** Un evento dispara múltiples acciones simultáneamente.
4. **RF-03V2:** Generación automática de reportes basada en eventos.

3.3 Patrón DAO (Data Access Object)

Ubicación en el código:

```

modelo/dao/ ├── IEmpleadoDAO.java ← Interfaz ├── EmpleadoDAO.java ← Implementación ├──
              ├── IUserioDAO.java ├── UsuarioDAO.java └─ ...

```

Definición:

El patrón DAO abstrae el acceso a la base de datos, proporcionando métodos de alto nivel para CRUD sin exponer detalles de implementación.

Implementación en SNAAR:

```

public interface IEmpleadoDAO {
    void crear(Empleado emp);
    Empleado buscarPorCedula(String cedula);
    List<Empleado> listarTodos();
    void editar(Empleado emp);
    void eliminar(String cedula);
    boolean existeCedula(String cedula);
}

public class EmpleadoDAO implements IEmpleadoDAO {
    private ConexionBD conexion;

    @Override
    public void crear(Empleado emp) {
        String sql = "INSERT INTO empleados (...) VALUES (...)";
        try (Connection conn = conexion.obtenerConexion();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, emp.getCedula());
            pstmt.setString(2, emp.getNombre());
            pstmt.executeUpdate();
        }
    }
}

```

Justificación:

1. **Abstracción:** La capa de servicios no conoce que usa JDBC o SQL.
2. **Cambio de BD:** Si se cambia a JPA/Hibernate, solo cambia `EmpleadoDAO`.

3. **Testing:** Se puede crear un `EmpleadoDAOMock` para pruebas unitarias.
4. **Reutilización:** El mismo DAO puede ser usado por múltiples servicios.

4. Patrones de Validación y Seguridad

4.1 Validación en Múltiples Capas

Controlador HTTP ↓ EmpleadoServicio.validar() ↓ ValidadorDatos (Util) ↓ Base de Datos (Restricciones SQL)

Justificación: La validación en múltiples niveles previene:

- Datos malformados en la UI
- Lógica de negocio inconsistente
- Violaciones de restricciones en BD

4.2 Cifrado de Contraseñas

```
public class CifradorContrasena {  
    public static String cifrar(String contrasena) {  
        // Usa algoritmo seguro (ej: BCrypt, PBKDF2)  
        return algoritmoSeguro(contrasena);  
    }  
}
```

Importante: Las contraseñas **NUNCA** se almacenan en texto plano. Se usa hash + salt para resistir ataques de fuerza bruta.

5. Generación de Credenciales

5.1 Credenciales Automáticas (RF-01.06)

```
public class GeneradorCredenciales {  
    public Credenciales generar(Empleado emp) {  
        String usuario = generarUsuario(emp.getNombre()); // ej: "jperez"  
        String contrasena = generarContrasenaSegura(); // ej: "Tx8#kL2m9!"  
        return new Credenciales(usuario, contrasena);  
    }  
}
```

Ventajas:

- **Seguridad:** Las credenciales son únicas y complejas.
- **Usabilidad:** El administrador no ingresa credenciales débiles.
- **Auditoría:** Se registra quién generó qué credenciales.

6. Reportes (RF-03V2)

6.1 Arquitectura de Reportes

```
ReporteServicio |— Consulta BD (historial) |— Procesa datos |— Genera estructura |—  
Notifica observadores ObservadorAuditoria → Registra generación ObservadorNotificacion →  
Notifica al usuario
```

Justificación:

- Los reportes son **generados bajo demanda**, no precalculados.
- El patrón Observer automáticamente notifica cuando se genera un reporte.
- Los datos son **auditados** para cumplimiento normativo.

7. Gestión de Sesiones y Autenticación (RF-02)

7.1 Flujo de Autenticación

1. Usuario envía credenciales ↓ 2. AuthController.iniciarSesion() ↓ 3. AuthServicio.autenticar() ↓ 4. UsuarioDAO.buscarPorUsuario() ↓ 5. Validación de contraseña con CifradorContrasena ↓ 6. Si es válido: Crear sesión (Sesion.java) ↓ 7. Si no: Registrar intento fallido (auditoría)

7.2 Cambio de Contraseña (RF-02.02)

```
public class AuthServicio {  
    public void cambiarContrasena(String usuario, String antiguo, String nuevo) {  
        // 1. Validar que la contraseña antigua sea correcta  
        // 2. Validar que la nueva cumpla políticas  
        // 3. Actualizar en BD  
        // 4. Notificar observadores  
    }  
}
```

7.3 Recuperación de Contraseña (RF-02.03)

```
public class AuthServicio {  
    public void recuperarContrasena(String usuario, String correo) {  
        // 1. Validar que usuario y correo coincidan  
        // 2. Generar nueva contraseña temporal  
        // 3. Enviar por correo  
        // 4. Registrar en auditoría  
    }  
}
```

Diseño: La lógica de autenticación está **centralizada** en `AuthServicio`. Cada operación es **auditada** para seguridad. Las contraseñas son **generadas y cifradas** automáticamente.

8. Ventajas de la Arquitectura Propuesta

Aspecto	Ventaja	Justificación
Mantenibilidad	Cambios localizados	Separación de responsabilidades
Escalabilidad	Fácil agregar features	Cada capa es independiente
Testabilidad	Tests aislados	Interfaces y DAOs mockeables
Seguridad	Múltiples niveles	Validación y auditoría en cada capa
Auditoría	Historial completo	Patrón Observer registra eventos
Reutilización	Código DRY	Servicios reutilizables
Performance	Conexión pooling	Gestión eficiente de BD

9. Decisiones Tecnológicas

9.1 ¿Por qué Java + Servlet + JSP?

- **Robustez:** Java es enterprise-grade y altamente confiable.
- **Servlet API:** Estándar de la industria para aplicaciones web.
- **JSP:** Facilita mezclar lógica y presentación (aunque separamos en servicios).
- **JDBC:** Control directo sobre queries SQL.

9.2 ¿Por qué no usar frameworks como Spring o Hibernate?

Contexto: Este es un prototipo académico que debe demostrar patrones de diseño.

- **Spring:** Añadiría abstracción pero ocultaría los patrones.
- **Hibernate:** JPA es más abstracto pero menos educativo.
- **JDBC puro:** Permite entender cómo funciona realmente la persistencia.

Para producción, se recomienda:

- Spring Boot (gestión de dependencias y configuración)
- Spring Data JPA (abstracción de BD)
- Spring Security (autenticación/autorización)
- Thymeleaf (templates más seguros que JSP)

9.3 ¿Por qué Maven?

- Gestión automática de dependencias.
- Build reproducible.
- Estándar en proyectos Java enterprise.

10. Riesgos y Mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
SQL Injection	Media	Alto	Usar PreparedStatement
Contraseña débil	Alta	Alto	Validación y generación automática
Falta de auditoría	Media	Alto	Patrón Observer en cada operación
Performance con BD	Baja	Medio	Connection pooling e índices
Falta de logs	Alta	Medio	Logging en servicios y DAOs

11. Conclusión

La arquitectura de **tres capas** combinada con patrones **Command**, **Observer** y **DAO** proporciona una base sólida para el sistema SNAAR. Cada decisión ha sido justificada considerando:

1. **Separación de responsabilidades:** Cada capa tiene un propósito claro.
2. **Mantenibilidad:** Cambios futuros son fáciles de implementar.
3. **Testabilidad:** Componentes pueden probarse aisladamente.
4. **Seguridad:** Múltiples capas de validación y auditoría.
5. **Escalabilidad:** Nuevas características se agregan sin rediseño.

El presente documento valida que el prototipo SNAAR cumple con los estándares de arquitectura empresarial y patrones de diseño reconocidos en la industria.

12. Referencias y Lecturas Recomendadas

- **Gang of Four (GoF) Design Patterns:** *Design Patterns: Elements of Reusable Object-Oriented Software*
- **Enterprise Integration Patterns:** Hohpe & Woolf
- **Clean Architecture:** Robert C. Martin
- **The Pragmatic Programmer:** Hunt & Thomas
- **Java Design Patterns:** <https://refactoring.guru/design-patterns/java>

SNAAR TekMess v1.0

Documento técnico de arquitectura y patrones de diseño

Análisis y Diseño de Software | Junio 2026

Este documento puede ser impreso o exportado a PDF manteniendo su formato profesional.